MCP Server Kubernetes + Claude AI Integration (On-Prem Setup)

Overview:
This setup integrates:
- mcp-server-kubernetes
- Claude
- Kubernetes cluster (kubeadm – 4 nodes)

Purpose:
Control Kubernetes using natural language via AI

Architecture:

```
Claude CLI (AI Agent)
      ↓ (stdio)
MCP Server (Node.js)
      ↓
Kubernetes Cluster (via kubeconfig)
```

Prerequisites
- Ubuntu 20.04 / 22.04 server
- Kubernetes cluster (kubeadm)
- kubectl configured
- Internet access

Step 1: Clone Repository
```
sudo su
cd ~
git clone https://github.com/Flux159/mcp-server-kubernetes.git
cd mcp-server-kubernetes
```

Step 2: Install Bun
```
curl -fsSL https://bun.sh/install | bash
source ~/.bashrc
bun –version
```

Install Node: above 20.X.X version
```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
apt install nodejs -y
```

Step 3: Install Dependencies
```
bun install
```
If slow:
```
export BUN_INSTALL_CACHE_DIR=/tmp/bun-cache
```

Step 4: Build Project
```
bun run build
```
This generates: $ tsc && shx chmod +x dist/*.js

Step 5: Configure Kubernetes Access (if not)

```
mkdir -p ~/.kube
sudo cp /etc/kubernetes/admin.conf ~/.kube/config
sudo chown $USER:$USER ~/.kube/config
kubectl get nodes
```

Permission error:

```
sudo chown -R $USER:$USER ~/mcp-server-kubernetes
```

Step 6: Install Claude CLI

```
curl -fsSL https://claude.ai/install.sh | bash
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
claude –help
```

Step 7: Login (IMPORTANT – On-Prem)

Use API Key method (recommended)

Create API Key

Open in browser:

https://console.anthropic.com

Generate API key

Login via CLI

claude login OR claude

Select:

2. Anthropic Console account (API usage)

paste that API key here , you will get succuessfully login prompt

Step 8: Configure MCP Server (Do it on for clouad Desktop only)

claude config edit

```
{
  "mcpServers": {
    "kubernetes": {
      "command": "node",
      "args": ["/home/<your-user>/mcp-server-kubernetes/dist/index.js"],
      "env": {
        "KUBECONFIG": "/home/<your-user>/.kube/config"
      }
    }
  }
}
```

Replace <your-user> accordingly

Step 9: Start Claude

```
claude
```

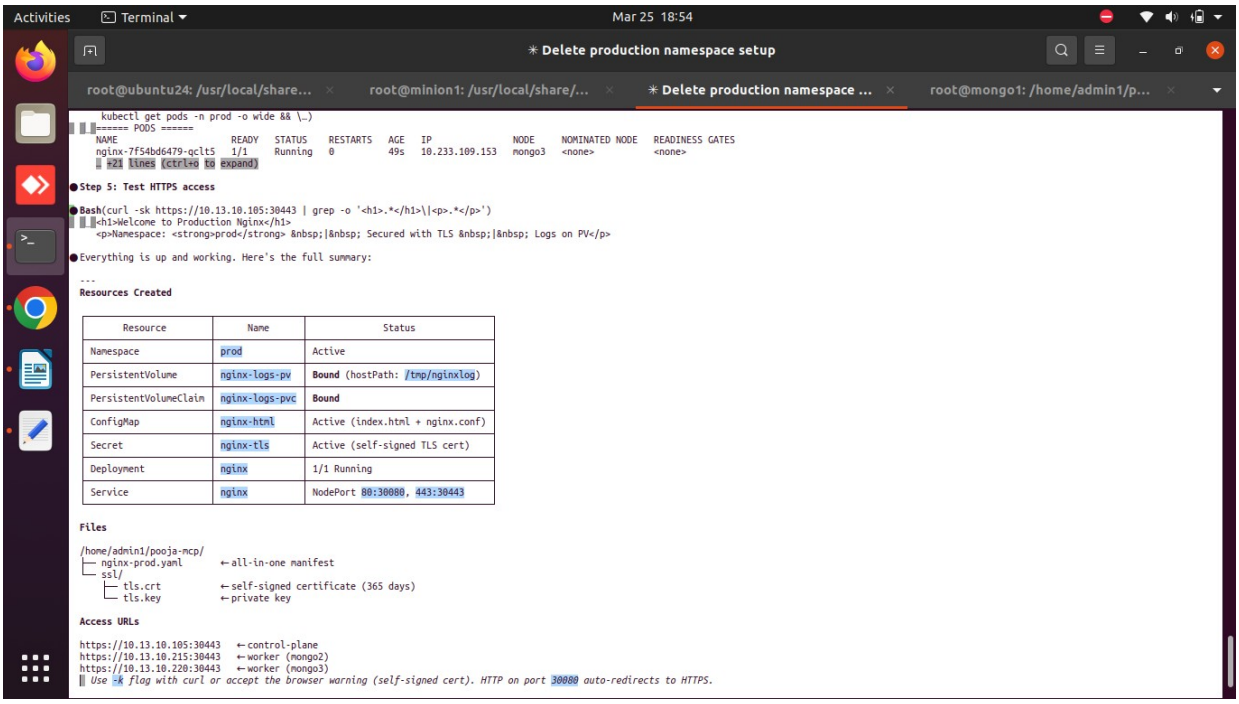
Step 10: Test Integration

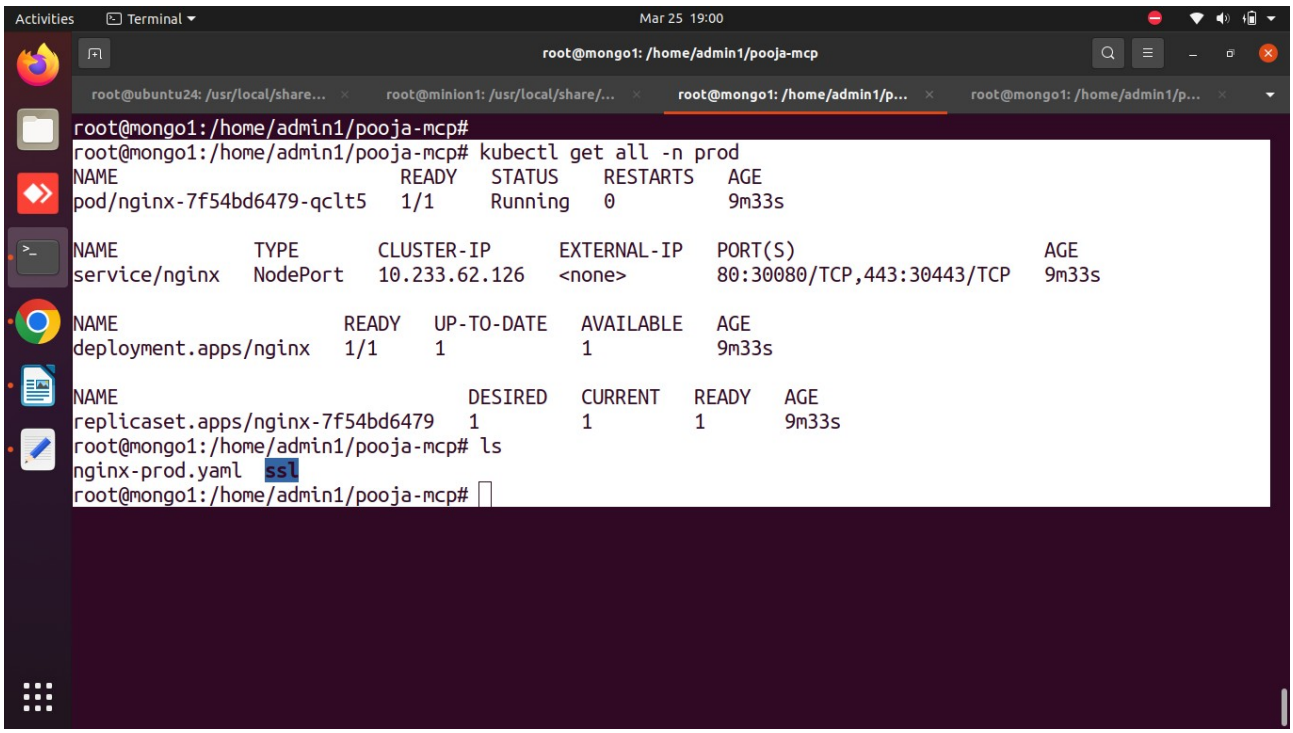
List all Kubernetes pods, Show all nodes in my cluster

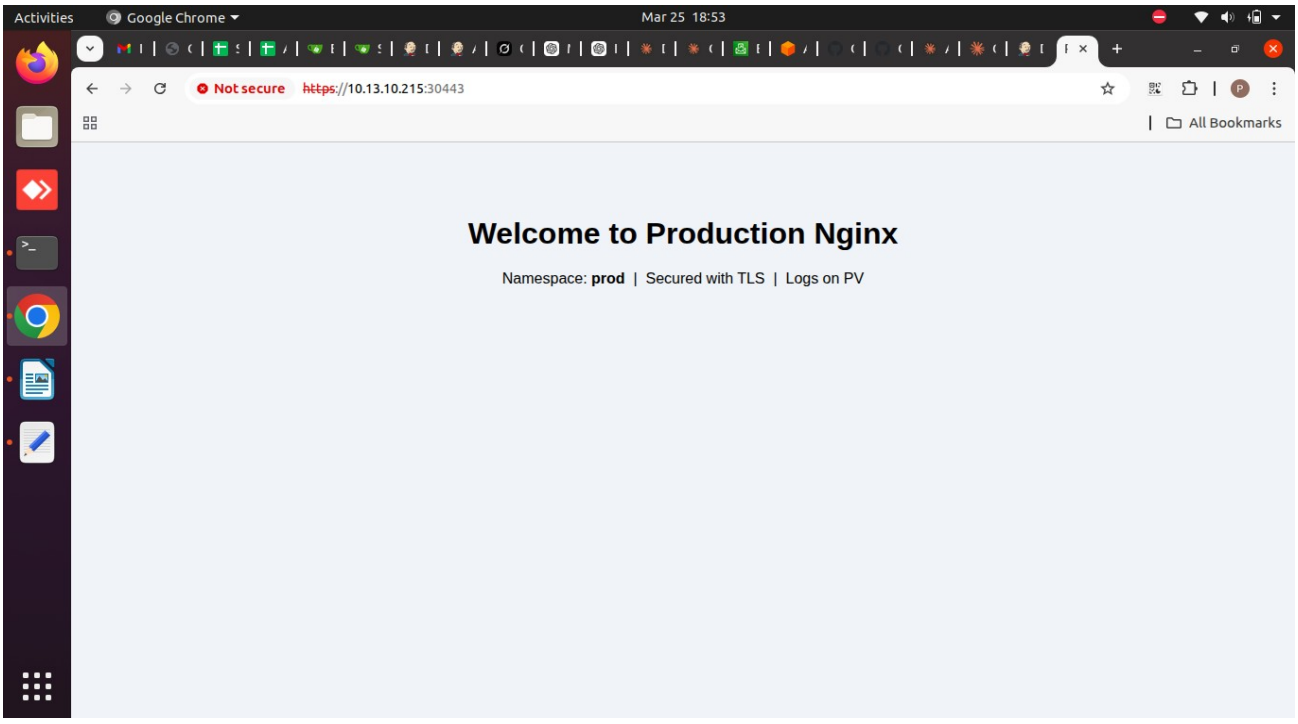
Example Prompt (Nginx Deployment):
Create a complete Kubernetes setup in my cluster with the following requirements:
1. Create a namespace called "prod"
2. Create a PersistentVolume (PV):
   - Use local storage
   - Path: /tmp/nginxlog
   - Capacity: 1Gi
   - Access mode: ReadWriteOnce
3. Create a PersistentVolumeClaim (PVC):
   - Namespace: prod
   - Request: 1Gi
   - Access mode: ReadWriteOnce
4. Create a ConfigMap:
   - Namespace: prod
   - Name: nginx-html
   - Add custom index.html content like "Welcome to Production Nginx"
5. Create an Nginx Deployment:
   - Namespace: prod
   - Use nginx:latest image
   - Mount PVC to /var/log/nginx
   - Mount ConfigMap to /usr/share/nginx/html
6. Expose the deployment using a NodePort service Add Ssl for webiste create ssl direcrtoy in same folder..
7. Ensure all resources are properly linked and working..
After creating, verify all resources and show me:
- pods – pv – pvc - services

procced with yes only.

Verify Resources
kubectl get all -n prod
kubectl get pv
kubectl get pvc -n prod





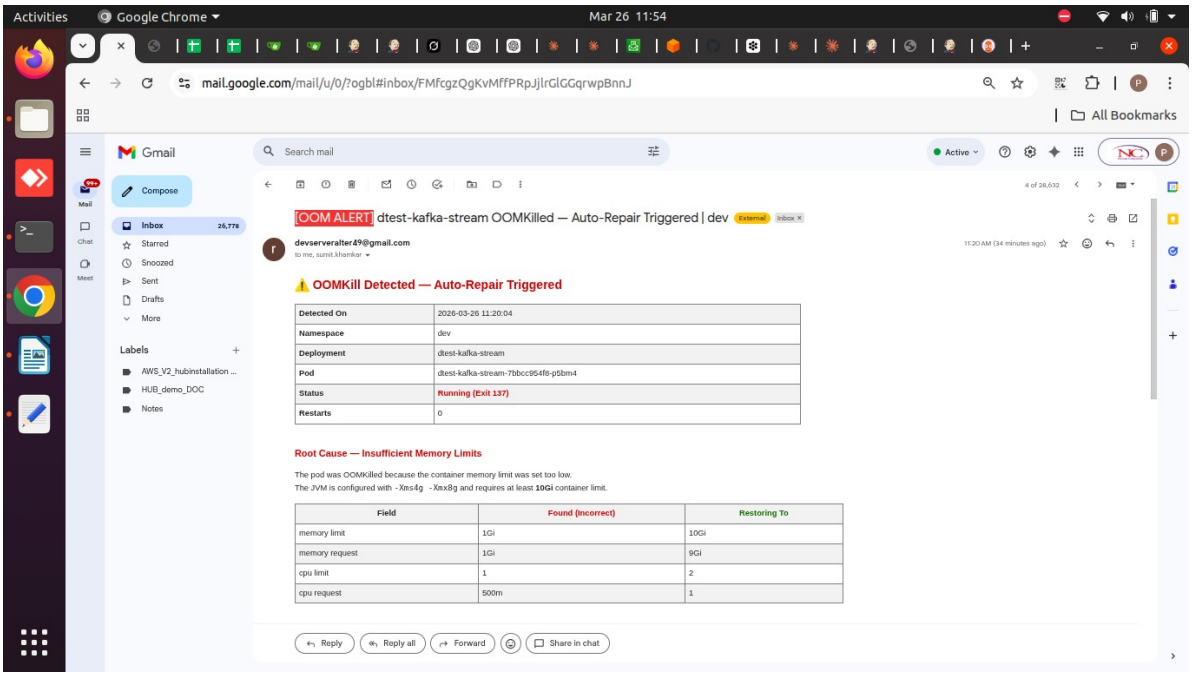
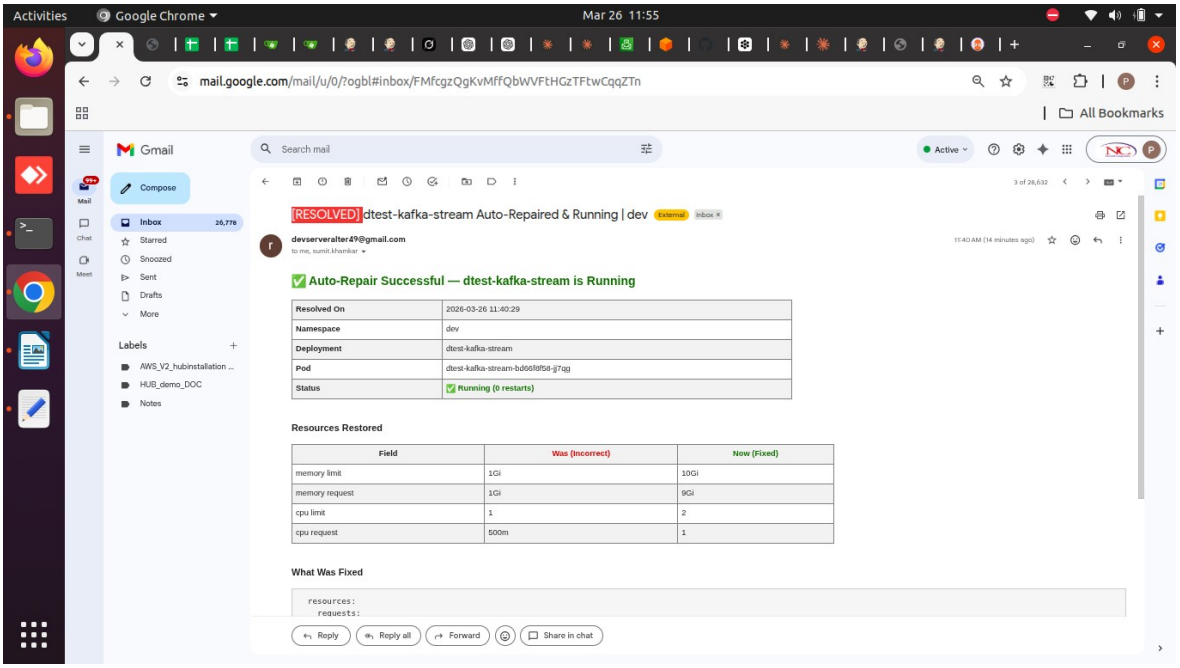


Security Best Practice:
Avoid using admin.conf in production.
Create limited access:
kubectl create serviceaccount mcp-user
kubectl create clusterrolebinding mcp-binding \
--clusterrole=view \
--serviceaccount=default:mcp-user

Production Improvements

- Use Nginx as reverse proxy
- Add HTTPS (Let’s Encrypt)
- Use restricted RBAC
- Monitor via Prometheus/Grafana

Key Concepts

Component	Role
Claude	AI agent
MCP Server	Middleware
Kubernetes	Execution layer

Using Claude AI, I created an automated script that monitors the entire Kubernetes cluster, detects pod-level errors, and triggers email notifications. The system not only alerts on issues but also attempts automated fixes and sends a follow-up email once the issue is resolved.